# Deformation Transfer for Triangle Meshes

Pramook Khungurn

March 9, 2026

This note is written as I read the "deformation transfer for triangle meshes" paper by Sumner and Popovic [SP04]. This is a classic paper that starts the whole field of deformation transfer [RdMA21].

## 1 Introduction

- The input has three triangular meshes.
  - The *source* mesh $S$.
  - The *deformed source* mesh $\widetilde{S}$.
  - The *target* mesh $T$.
- The constraint between the input meshes is that $S$ and $\widetilde{S}$ should have the same topology: same number of vertices and triangles. The target mesh does not have to have the same topology.
- We want to create a new triangular mesh $\widetilde{T}$ that has the same topology as $T$. We want $\widetilde{T}$ to be $T$ with the same global, continuous, deformation of shape that was applied to get $\widetilde{S}$ from $S$.
- Some more notations.
  - Let $S$ and $\widetilde{S}$ have $n$ vertices.
  - Let $|S|$ denote the number of triangles in $S$.
  - Let us denote the vertices of $S$ by $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n$.
  - Let us denote the vertices of $\widetilde{S}$ by $\widetilde{\mathbf{v}}_1, \widetilde{\mathbf{v}}_2, \ldots, \widetilde{\mathbf{v}}_n$.
  - Let $T$ and $\widetilde{T}$ have $m$ vertices.
  - Let $|T|$ denote the number of triangles in $T$.
  - Let us denote the vertices of $T$ by $\mathbf{t}_1, \mathbf{t}_2, \ldots, \mathbf{t}_m$.
  - Let us denote the vertices of $\widetilde{T}$ by $\widetilde{\mathbf{t}}_1, \widetilde{\mathbf{t}}_2, \ldots, \widetilde{\mathbf{t}}_m$.
- To take into account the differences between topology of $S$ and $T$, the user supplies a mapping $M$ between the source and the target triangles.

  $$M = \{(s_1, t_1), (s_2, t_2), \ldots, (s_{|M|}, t_{|M|})\}.$$

  A pair $(s_i, t_i)$ means that the target triangle with index $t_i$ should deform like the source triangle with index $s_i$.
- In general, $M$ is a many-to-one mapping.
- Creating the mapping $M$ is a tedious task if it were to be done manually. The paper proposes a method to create it semi-automatically. We will talk about it later.

## 2 First Formulation

- The *source deformation* that gets us from $S$ to $\widetilde{S}$ is a collection of affine transformations tabulated for each triangle in $S$.
  - We cannot compute this affine transformation from the vertex positions of the triangles in $S$ and $\widetilde{S}$. This is because the changes in vertex positions do not determine how the space perpendicular to the triangle deforms.
  - To resolve the problem, we add a fourth vertex in the direction perpendicular to the triangle. Let $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_3$ be the vertices of the triangle in $S$. The fourth vertex $\mathbf{v}_4$ is given by:

    $$\mathbf{v}_4 = \mathbf{v}_1 + \frac{(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)}{\sqrt{\|(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)\|}}.$$

    We also perform the same computation for the vertices $\widetilde{\mathbf{v}}_1$, $\widetilde{\mathbf{v}}_2$, and $\widetilde{\mathbf{v}}_3$ in $\widetilde{S}$.
  - Note that the vector that we add to $\mathbf{v}_1$ to get $\mathbf{v}_4$ is not a unit vector. Its length is proportional to the square root of the area of the triangle. It scales proportionally to the length of the triangle edges.
  - The affine transformation is defined by a $3 \times 3$ matrix $Q$ and a displacement vector $\mathbf{d} \in \mathbb{R}^3$. We will write it as $Q + \mathbf{d}$.
  - We want the transformation to be so that

    $$Q\mathbf{v}_i + \mathbf{d} = \widetilde{\mathbf{v}}_i$$

    for $i = 1$, 2, 3, and 4.
  - If we subtract the first equation from the others, we have that

    $$\begin{aligned} Q(\mathbf{v}_2 - \mathbf{v}_1) &= \widetilde{\mathbf{v}}_2 - \widetilde{\mathbf{v}}_1, \\ Q(\mathbf{v}_3 - \mathbf{v}_1) &= \widetilde{\mathbf{v}}_3 - \widetilde{\mathbf{v}}_1, \\ Q(\mathbf{v}_4 - \mathbf{v}_1) &= \widetilde{\mathbf{v}}_4 - \widetilde{\mathbf{v}}_1. \end{aligned}$$

    Putting the above equations into a matrix, we have that

    $$Q \begin{bmatrix} \mathbf{v}_2 - \mathbf{v}_1 & \mathbf{v}_3 - \mathbf{v}_1 & \mathbf{v}_4 - \mathbf{v}_1 \end{bmatrix} = \begin{bmatrix} \widetilde{\mathbf{v}}_2 - \widetilde{\mathbf{v}}_1 & \widetilde{\mathbf{v}}_3 - \widetilde{\mathbf{v}}_1 & \widetilde{\mathbf{v}}_4 - \widetilde{\mathbf{v}}_1 \end{bmatrix}.$$

    If we let

    $$\begin{aligned} V &= \begin{bmatrix} \mathbf{v}_2 - \mathbf{v}_1 & \mathbf{v}_3 - \mathbf{v}_1 & \mathbf{v}_4 - \mathbf{v}_1 \end{bmatrix}, \\ \widetilde{V} &= \begin{bmatrix} \widetilde{\mathbf{v}}_2 - \widetilde{\mathbf{v}}_1 & \widetilde{\mathbf{v}}_3 - \widetilde{\mathbf{v}}_1 & \widetilde{\mathbf{v}}_4 - \widetilde{\mathbf{v}}_1 \end{bmatrix}, \end{aligned}$$

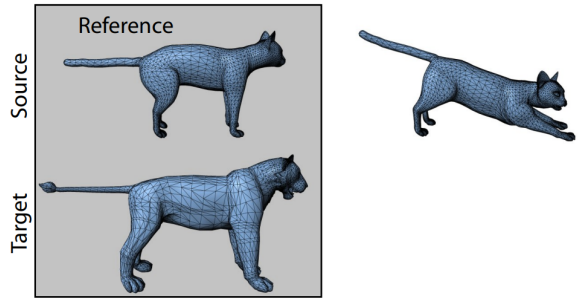
    we have that

    $$QV = \widetilde{V}.$$

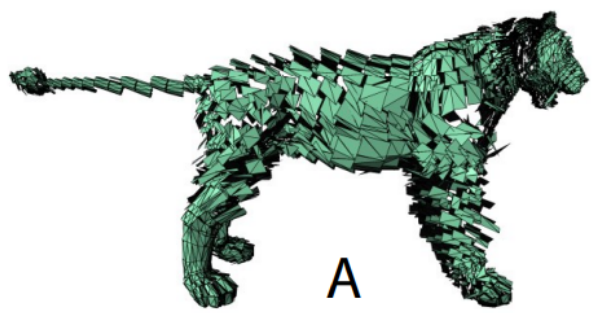
    This gives us

    $$Q = \widetilde{V}V^{-1}.$$
  - Once we have $Q$, we can use it to solve for $\mathbf{d} = \widetilde{\mathbf{v}}_1 - Q\mathbf{v}_1$.
  - For each triangle $j$, we compute $\mathtt{S}_j = Q_j + \mathbf{d}_j$ for $j = 1, 2, \ldots, |S|$.
- Let's try to understand the effect of the source transformation on the target mesh, assuming that we already have the mapping in place.

- First, let's get ourselves the source mesh (top left), the deformed source mesh (bottom left), and the target mesh (top right).

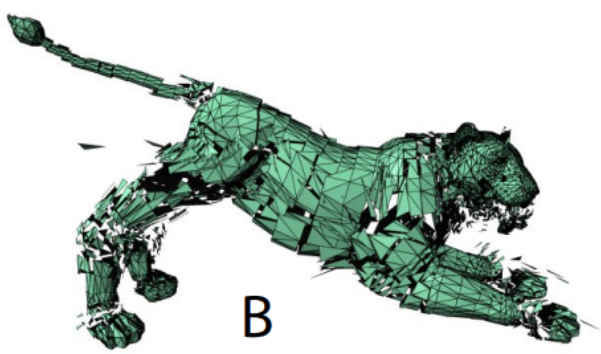



- If we apply only the linear part of the source deformation, $Q_j$, to the target triangles, we get something like this.




  We see here that the triangles in the target mesh change their orientations and sizes, but the positions are not changed. (Recall that everything is anchored to $\mathbf{v}_1$.)

- If we also apply the translation $\mathbf{d}_j$ to the target triangles as well, we get something like this.




  We see that the triangles also change their positions. However, the mesh is no longer connected.

- So, in order not to tear up the target mesh, we need to make sure that the triangles are connected after transformations.
  - Let us say that the target deformation is $\mathtt{T}_j = R_j + \mathbf{e}_j$ for $j = 1, 2, \ldots, |T|$.
    - Here, $R_j$ is the analogue of $Q_j$, and $\mathbf{e}_j$ is the analogue of $\mathbf{d}_j$.
  - We require that shared vertices be transformed to the same position. In other words, let $\mathbf{w}_i$ be a

vertex in the target mesh, and let $p(\mathbf{w}_i)$ be the set of triangles that contains $\mathbf{w}_i$. We require that

$$R_j\mathbf{w}_i + \mathbf{e}_j = R_k\mathbf{w}_i + \mathbf{e}_k$$

for all $j, k \in p(\mathbf{w}_i)$.

- Deformation transfers solve an optimization problem to make the target deformation as close as possible to the source deformation while maintaining the connectedness above.

$$\min_{\mathtt{T}_1,\mathtt{T}_2,\ldots,\mathtt{T}_{|T|}} \sum_{j=1}^{|M|} \|\mathtt{T}_{t_j} - \mathtt{S}_{s_j}\|_F^2$$

$$\text{subject to: } R_j\mathbf{w}_i + \mathbf{e}_j = R_k\mathbf{w}_i + \mathbf{e}_k \text{ for all } j, k \in p(\mathbf{w}_i) \text{ for all vertices } \mathbf{w}_i.$$

Recall that $\mathtt{S}_{s_j}$ and $\mathtt{T}_{t_j}$ are affine transformations in 3D, and this means that they can be represented by $4 \times 4$ matrices. So, the Frobenius norm of the difference between the two is well defined.

# 3 Vertex Formulation

- The above formulation is a constrained quadratic optimization problem with linear constraints. While this is a convex optimization problem that has a solution, it is inefficient. (Honestly, I also don't know how to do it myself.)
- There is a much better formulation that casts the problem in terms of the output vertex positions.
  - We define the target deformation in terms of the output vertex positions.
  - We solve directly for the output vertex positions so as to minimize the difference between the source and the target deformations.
- To do this, for each target triangle, we add a fourth vertex. Then, we have that

$$R = \widetilde{W}W^{-1}.$$

  - The elements of $W^{-1}$ are known: they are the vertices of the target mesh.
  - The elements of $\widetilde{W}$ are unknown: they are the vertices of the deformed target mesh.
  - So, the elements of $R$ are linear combinations of the coordinates of the unknown deformed vertices.
- With the above identity, we can rewrite the minimization problem as follows:

$$\min_{\widetilde{\mathbf{w}}_1,\widetilde{\mathbf{w}}_2,\ldots,\widetilde{\mathbf{w}}_m} \sum_{j=1}^{|M|} \|\mathtt{T}_{t_j} - \mathtt{S}_{s_j}\|_F^2$$

- We can rewrite the above minimization problem as a least squares problem.

$$\min_{\widetilde{\mathbf{w}}_1,\widetilde{\mathbf{w}}_2,\ldots,\widetilde{\mathbf{w}}_m} \|\mathbf{c} - A\widetilde{\mathbf{w}}\|_2^2.$$

  Here:

  - $\widetilde{\mathbf{w}}$ is the big vector of unknown deformed vertex positions.
  - $\mathbf{c}$ is a vector containing entries from the source transformations.
  - $A$ is a large matrix that relates $\widetilde{\mathbf{w}}$ to $\mathbf{c}$.

  The optimal solution is given by

$$\widetilde{\mathbf{w}} = (A^TA)^\dagger A^T\mathbf{c},$$

  which is standard knowledge.
- In practice, we need to compute $A$. Then, we can factor $A^TA$ so that it becomes easy to invert, like with an LU or a Cholesky factorization or something like that. Then, we can solve for $\widetilde{\mathbf{w}}$.

# 4 Correspondence

- We need to discuss how to determine $M$, the mapping between the source and the target triangles.
- For this, we need help from the user.
  - The user selects $N$ pairs of source and target vertices that correspond to each other.
  - Let us call these pairs $(a_1, b_1), (a_2, b_2), \ldots, (a_N, b_N)$.
  - A pair $(a_i, b_i)$ indicates that the source vertex $\mathbf{v}_{a_i}$ should correspond to the target vertex $\mathbf{w}_{b_i}$.
- The mapping is then computed with an iterated closest point algorithm with regularization.
- The algorithm performs the following steps:
  1. It deforms the source mesh to the target mesh.
  2. It computes the triangle correspondence by searching for pairs of source and target triangles whose centroids are in close proximity.
- To deform the source mesh to the target mesh, we solve a minimization problem that is similar to the one we used in the last section.
  - However, the objective function is designed to deform one mesh *into* the other rather than deforming it *like* how the other deforms.
  - A pair of corresponding vertices indicates that, after deformation, the source vertex should match the location of the target vertex. Each pair becomes an optimization constraint.
  - The objective function also contains multiple terms.
    - One that enforces deformation smoothness.
    - One that prevents over smoothing.
    - One that moves the source vertices to the target vertices.
- Let us set up the transformation problem.
  - We wish to optimize the vertex positions of the deformed mesh $\widetilde{S}$ so that it is as close as possible to the target mesh $T$.
  - The transformation we wish to optimize is $\mathtt{S}_j = Q_j + \mathbf{d}_j$ for $j = 1, 2, \ldots, |S|$.
  - Again, we have that

    $$Q_j = \widetilde{V}_j V_j^{-1}.$$

    Here, $\widetilde{V}_i$ is unknown because it is a function of the unknown vertex positions of $\widetilde{S}$. On the other hand, $V_i$ is known because it is a function of the known vertex positions of $S$.
- The *deformation smoothness loss* $E_S$ requires that the transformations for adjacent triangles should be similar.

  $$E_S = \sum_{j=1}^{|S|} \sum_{k \in \mathrm{adj}(j)} \|\mathtt{S}_j - \mathtt{S}_k\|_F^2$$

  where $\mathrm{adj}(j)$ is the set of triangles that are adjacent to triangle $j$.

- The *deformation identity loss* $E_I$ requires that all the transformations should be close to the identity transformation.

$$E_I = \sum_{j=1}^{|S|} \|\mathtt{S}_j - I\|_F^2$$

  Its purpose is to prevent the deformation smoothness loss from generating a drastic change in the shape of the mesh in order to achieve optimal smoothness (such as collapsing the mesh into a single point).

- The *closest valid point term* $E_C$ requires that the deformed source vertex should be equal to the closest valid point on the target mesh.

$$E_i = \sum_{i=1}^{n} \|\widetilde{\mathbf{v}}_i - \mathbf{c}_i\|^2.$$

  Here, $\mathbf{c}_i$ is the closest "valid point" on the target mesh to the deformed source vertex $\widetilde{\mathbf{v}}_i$.

  - A point on the target mesh is considered "valid" for $\widetilde{\mathbf{v}}_i$ if its normal vector makes an angle of less than 90° with the normal vector of $\widetilde{\mathbf{v}}_i$.
  - A grid-based spatial sorting algorithm accelerates the closest point computation for this loss.

- The optimization problem for correspondence is as follows.

$$\begin{aligned} &\min_{\widetilde{\mathbf{v}}_1, \widetilde{\mathbf{v}}_2, \ldots, \widetilde{\mathbf{v}}_n} \lambda_S E_S + \lambda_I E_I + \lambda_C E_C \\ &\text{subject to: } \widetilde{\mathbf{v}}_{a_i} = \mathbf{w}_{b_i} \text{ for all } i = 1, 2, \ldots, N. \end{aligned}$$

  Here, $\lambda_S$, $\lambda_I$, and $\lambda_C$ are weights of the loss terms.

- Note that the constraints basically reduce the number of unknowns, so it is a least-square problem with no constraints.

- The minimization problem is solved in two phases.

  - In the first phase, we set $w_S = 1.0$, $w_I = 0.001$, and $w_C = 0.0$, ignoring the closest point term.
  - In the second phase, we solve the same problem, increasing $w_C$ each time and updating the closest points after each iteration. The weights $w_S$ and $w_I$ remain the same, but $w_C$ is increased in four steps from 1.0 to 5000.0.

- Once the source mesh has been deformed to the shape of the target mesh, we compute triangle correspondences by comparing the centroids of the deformed source and target triangles.

  - Two triangles are "compatible" if the centroids are within a certain threshold of each other and the angle between their normals is less than 90°.
  - For each triangle in the deformed source, we compute the closest compatible triangle in the target mesh, and add the pair to the mapping $M$.
  - For each triangle in the target, we compute the closest compatible triangle in the deformed source mesh, and add the pair to the mapping $M$.
  - A target triangle may correspond to multiple source triangles and vice versa.

# 5 Vertex Formulation Details

- To repeat optimization problem for the deformation transfer is as follows.

$$\min_{\widetilde{\mathbf{w}}_1, \widetilde{\mathbf{w}}_2, \ldots, \widetilde{\mathbf{w}}_m} \|\mathbf{c} - A\widetilde{\mathbf{w}}\|_2^2.$$

  However, we didn't actually specify what these variables are.

- The first thing to consider that is how many unknowns are there. From the statement, it seems that there are $3m$ unknowns. However, this is not true.
  - Remember that for each triangle in the deformed target mesh, we need to add a fourth vertex.
  - We have seen so far that, in other $S$, $\widetilde{S}$, and $T$, the fourth vertex is a function of the three vertices of the triangle.
  - However, the vertices of $\widetilde{T}$ are unknown. If we add a function of unknowns and compute with it, the problem becomes much more complicated to the extent that it cannot be casted as a least-squares problem.
  - So, we need to add an unknonw point for each of the triangles instead.
  - As a result, there are $3(m + |T|)$ unknowns.
  - Therefore, $\widetilde{\mathbf{w}}$ is a vector of length $3(m + |T|)$.

- Next, let's talk about $\mathbf{c}$. How many components that it has?
  - Recall that $\|\mathbf{c} - A\widetilde{\mathbf{w}}\|_2^2$ is supposed to be equal to $\sum_{j=1}^{|M|} \|\mathtt{T}_{t_j} - \mathtt{S}_{s_j}\|_F^2$.
  - As a result, there are $M$ pairs of matrices to compare.
  - We have that $T_{t_j}$ and $S_{s_j}$ are $4 \times 4$ matrices. However, the 4th row is fixed to $(0, 0, 0, 1)$. So, we only need to compare the first three rows, which has 12 numbers.
  - So, $\mathbf{c}$ has $12|M|$ components.

- As result, $A$ is a $12|M| \times 3(m + |T|)$ matrix. This matrix is mostly sparse.

# 6 Correspondence Details

Note that material in this section is considered standard knowledge in optimization. However, I will include it here so that I have a nice reference when I implement stuffs.

## 6.1 Combining Multiple Losses

- Suppose that we have a loss function that has multiple terms.

$$E = \sum_{i=1}^{k} \lambda_i E_i$$

  where each term $E_i$ is of the form

$$E_i = \|\mathbf{b}_i - A_i\mathbf{x}\|^2,$$

  and $\lambda_i$ are positive weights.

- We have that

$$\begin{aligned} E_i &= (A_i\mathbf{x} - \mathbf{b}_i)^T (A_i\mathbf{x} - \mathbf{b}_i) \\ &= \mathbf{x}^T A_i^T A_i \mathbf{x} - 2\mathbf{x}^T A_i^T \mathbf{b}_i + \mathbf{b}_i^T \mathbf{b}_i. \end{aligned}$$

  As a result,

$$\begin{aligned} E &= \sum_{i=1}^{k} \lambda_i (\mathbf{x}^T A_i^T A_i \mathbf{x} - 2\mathbf{x}^T A_i^T \mathbf{b}_i + \mathbf{b}_i^T \mathbf{b}_i) \\ &= \mathbf{x}^T \left( \sum_{i=1}^{k} \lambda_i A_i^T A_i \right) \mathbf{x} - 2\mathbf{x}^T \left( \sum_{i=1}^{k} \lambda_i A_i^T \mathbf{b}_i \right) + \sum_{i=1}^{k} \lambda_i \mathbf{b}_i^T \mathbf{b}_i. \end{aligned}$$

  The loss function is minimized when $\mathbf{x}$ is the solution to the *normal equation*:

$$\left( \sum_{i=1}^{k} \lambda_i A_i^T A_i \right) \mathbf{x} = \left( \sum_{i=1}^{k} \lambda_i A_i^T \mathbf{b}_i \right).$$

  The solution is given by:

$$\mathbf{x} = \left( \sum_{i=1}^{k} \lambda_i A_i^T A_i \right)^{\dagger} \left( \sum_{i=1}^{k} \lambda_i A_i^T \mathbf{b}_i \right).$$

## 6.2 Dealwing with Constraints

- Note that the vector $\mathbf{x}$ in the previous section is the vector of unknown vertex positions of the deformed source mesh $\widetilde{\mathbf{v}}$, which has $3(n + |S|)$ components.
- Nevertheless, not all of the above components are free variables. The constraints are that $\widetilde{\mathbf{v}}_{a_i} = \mathbf{w}_{b_i}$ for all $i = 1, 2, \ldots, N$.
- As a result, there are only $3(n + |S| - N)$ free variables.
- Let us say that we partition $\widetilde{\mathbf{v}}$ into two blocks.

$$\widetilde{\mathbf{v}} = \begin{bmatrix} \widetilde{\mathbf{v}}_\star \\ \widetilde{\mathbf{v}}_\bullet \end{bmatrix}$$

  where the first block $\widetilde{\mathbf{v}}_\star$ consists of free variables, and the second block $\widetilde{\mathbf{v}}_\bullet$ consists of constrained values.

- Let

$$\begin{aligned} \mathtt{A} &= \sum_{i=1}^{k} \lambda_i A_i^T A_i, \\ \mathtt{b} &= \sum_{i=1}^{k} \lambda_i A_i^T \mathbf{b}_i. \end{aligned}$$

  We can divide both $\mathtt{A}$ and $\mathtt{b}$ into blocks as well.

$$\begin{aligned} \mathtt{A} &= \begin{bmatrix} \mathtt{A}_{\star\star} & \mathtt{A}_{\star\bullet} \\ \mathtt{A}_{\bullet\star} & \mathtt{A}_{\bullet\bullet} \end{bmatrix}, \\ \mathtt{b} &= \begin{bmatrix} \mathtt{b}_\star \\ \mathtt{b}_\bullet \end{bmatrix}. \end{aligned}$$

So,

$$
\begin{aligned}
E &= \begin{bmatrix}\widetilde{\mathtt{v}}_\star^T & \widetilde{\mathtt{v}}_\bullet^T\end{bmatrix}\begin{bmatrix}\mathtt{A}_{\star\star} & \mathtt{A}_{\star\bullet}\\ \mathtt{A}_{\bullet\star} & \mathtt{A}_{\bullet\bullet}\end{bmatrix}\begin{bmatrix}\widetilde{\mathbf{v}}_\star\\ \widetilde{\mathbf{v}}_\bullet\end{bmatrix} - 2\begin{bmatrix}\widetilde{\mathtt{v}}_\star^T & \widetilde{\mathtt{v}}_\bullet^T\end{bmatrix}\begin{bmatrix}\mathtt{b}_\star\\ \mathtt{b}_\bullet\end{bmatrix} + \sum_{i=1}^{k}\lambda_i\mathbf{b}_i^T\mathbf{b}_i\\
&= \widetilde{\mathtt{v}}_\star^T\mathtt{A}_{\star\star}\widetilde{\mathtt{v}}_\star + \widetilde{\mathtt{v}}_\star^T\mathtt{A}_{\star\bullet}\widetilde{\mathtt{v}}_\bullet + \widetilde{\mathtt{v}}_\bullet^T\mathtt{A}_{\bullet\star}\widetilde{\mathtt{v}}_\star + \widetilde{\mathtt{v}}_\bullet^T\mathtt{A}_{\bullet\bullet}\widetilde{\mathtt{v}}_\bullet - 2\widetilde{\mathtt{v}}_\star^T\mathtt{b}_\star - 2\widetilde{\mathtt{v}}_\bullet^T\mathtt{b}_\bullet + \sum_{i=1}^{k}\lambda_i\mathbf{b}_i^T\mathbf{b}_i.\\
&= \widetilde{\mathtt{v}}_\star^T\mathtt{A}_{\star\star}\widetilde{\mathtt{v}}_\star - 2\widetilde{\mathbf{v}}_\star^T\left[\mathtt{b}_\star - \frac{\mathtt{A}_{\star\bullet}\widetilde{\mathtt{v}}_\bullet + \mathtt{A}_{\bullet\star}^T\widetilde{\mathtt{v}}_\bullet}{2}\right] + C\\
&= \widetilde{\mathtt{v}}_\star^T\mathtt{A}_{\star\star}\widetilde{\mathtt{v}}_\star - 2\widetilde{\mathbf{v}}_\star^T\left[\mathtt{b}_\star - \mathtt{A}_{\star\bullet}\widetilde{\mathtt{v}}_\bullet\right] + C.
\end{aligned}
$$

As a result, the optimal solution is given by

$$\widetilde{\mathbf{v}}_\star = \mathtt{A}_{\star\star}^\dagger\left(\mathtt{b}_\star - \mathtt{A}_{\star\bullet}\widetilde{\mathbf{v}}_\bullet\right).$$

### 6.3 Deformation Smoothness Loss

- TODO

### 6.4 Deformation Identity Loss

- TODO

### 6.5 Closest Valid Point Loss

- TODO

## References


[RdMA21] R. A. Roberts, R. K. dos Anjos, A. Maejima, and K. Anjyo, *Deformation transfer survey*, Computers & Graphics **94** (2021), 52–61.

[SP04] R. W. Sumner and J. Popović, *Deformation transfer for triangle meshes*, ACM Trans. Graph. **23** (2004), no. 3, 399–405.